COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	07/28/26	**DATE SUBMITTED:**	07/28/26
NAME:	Aguila, Daniel C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A CODE:

main.cpp

HOA4P1 > main.cpp > main()

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include <stack>

int main(){
    //create an object for the stack
    std::stack<int> newStack;

    std::cout<< "Stack Empty?" << newStack.empty()<<std::endl;

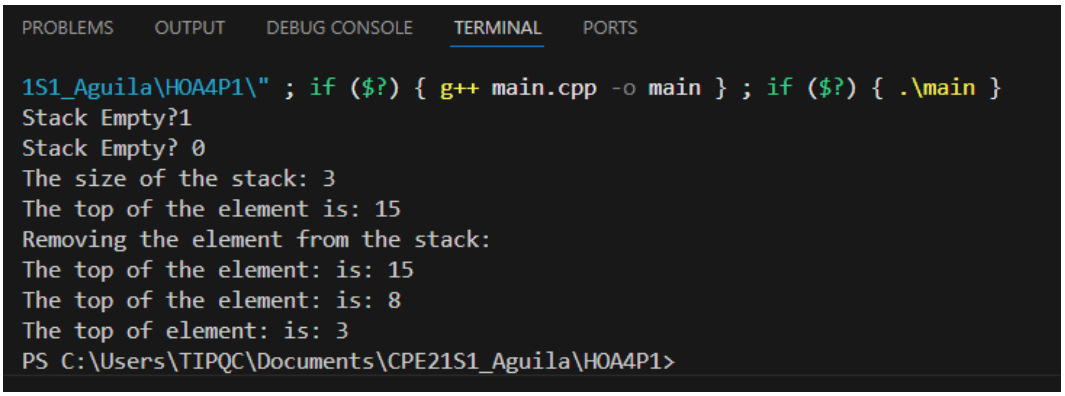
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? " <<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    std::cout<<"Removing the element from the stack: \n";

    std::cout<<"The top of the element: is: " <<newStack.top()<<std::endl;
    newStack.pop();
     std::cout<<"The top of the element: is: "<<newStack.top()<<std::endl;
    newStack.pop();
     std::cout<<"The top of element: is: "<<newStack.top()<<std::endl;
    newStack.pop();

    return 0;
}
```

ILO A OUTPUT:



ANALYSIS
On this code, this STL stack code directly shows how stack operations are handled without the need for manual index tracking or array management. The program creates a `std::stack<int>` object, checks if it's empty, then pushes values (`3`, `8`, `15`) onto it. It uses `.empty()` to verify state, `.size()` to get the number of elements, `.top()` to access the current top, and `.pop()` to remove elements one by one. Each step prints the stack's condition, making it clear how the stack changes after every operation. Compared to the array-based version, this code is shorter, safer, and easier to maintain because the STL automatically manages memory and prevents overflow or underflow errors. It highlights the efficiency and readability of using built-in containers for stack implementation.

ILO B CODE:

stack.cpp | size.cpp

HOA4P1 > size.cpp > pop()

```cpp
// Aguila, Daniel C. CPE21S1
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1;
int maxSize;
int newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main() {
    int choice;
    std::cout << "Enter the maximum number of elements for the stack: ";
    std::cin >> maxSize;

    while (true) {
        std::cout << "\nStack Operations Menu:\n";
        std::cout << " 1. PUSH\n 2. POP\n 3. TOP\n 4. isEMPTY\n 5. DISPLAY\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << (isEmpty() ? "Stack is empty\n" : "Stack is not empty\n");
            break;
            case 5: displayStack();
            break;
            default: std::cout << "Invalid choice.\n";
            break;
        }
    }
    return 0;
}
```

```
17  int main() {
22      while (true) {
28          switch (choice) {
42      }
43      return 0;
44  }
45
46  bool isEmpty() {
47      //create a condition to check whether the top element is equal to -1
48      return (top == -1);
49  }
50
51  bool isFull() {
52      //check if we reached the maximum capacity of the array;
53      return (top == maxSize - 1);
54  }
55
56  void push() {
57      //check if full -> if yes return an error
58      if (isFull()) {
59          std::cout << "Stack overflow.\n";
60          return;
61      }
62      std::cout << "Enter new value to push: ";
63      std::cin >> newData;
64      // increment the top(indext by 1) then assign the new data;
65      stack[++top] = newData;
66      std::cout << newData << " pushed into stack.\n";
67  }
68
69  void Top() {
70      //check if the stack is empty
71      if (isEmpty()) {
72          std::cout << "Stack underflow. No elements.\n";
73          return;
74      }
75      std::cout << "Top element: " << stack[top] << std::endl;
76  }
77
78  void pop() {
79      //check if the stack is empty
80      if (isEmpty()) {
81          std::cout << "Stack underflow. Nothing to pop.\n";
82          return;
83      }
84      //display the top value
85      std::cout << "Popping: " << stack[top] << std::endl;
```

```
82          return;
83      }
84      //display the top value
85      std::cout << "Popping: " << stack[top] << std::endl;
86      //decrement the top value from the stack
87      top--;
88  }
89
90  //display all elements
91  void displayStack() {
92      if (isEmpty()) {
93          std::cout << "Stack is empty.\n";
94          return;
95      }
96      std::cout << "Stack elements (top to bottom): ";
97      for (int j = top; j >= 0; j--) {
98          std::cout << stack[j] << " ";
99      }
100     std::cout << std::endl;
101 }
```

ILO B OUTPUT:

```
PS C:\Users\TIPQC\Documents\CPE21S1_Aguila> cd "c:\Users\TIPQC\Documents\CPE21S1_Aguila\HOA4P1\" ; if ($?) { g++ Size.cpp -o Size } ; if ($?) { .\Size }
Enter the maximum number of elements for the stack: 3

Stack Operations Menu:
 1. PUSH
 2. POP
 3. TOP
 4. isEMPTY
 5. DISPLAY
Enter choice: 1
Enter new value to push: 10
10 pushed into stack.

Stack Operations Menu:
 1. PUSH
 2. POP
 3. TOP
 4. isEMPTY
 5. DISPLAY
Enter choice: 1
Enter new value to push: 20
20 pushed into stack.

Stack Operations Menu:
 1. PUSH
 2. POP
 3. TOP
 4. isEMPTY
 5. DISPLAY
Enter choice: 1
Enter new value to push: 30
30 pushed into stack.

Stack Operations Menu:
 1. PUSH
 2. POP
 3. TOP
 4. isEMPTY
 5. DISPLAY
Enter choice: 1
Stack overflow.
```

ANALYSIS

In this code we tackled, the code demonstrates a clear and functional implementation of a **stack using arrays in C++** where each operation is handled through a dedicated function. It begins by asking the user to define the stack's maximum size, then repeatedly offers a menu for performing operations such as `PUSH`, `POP`, `TOP`, `isEMPTY`, and `DISPLAY`. The logic correctly checks for overflow and underflow conditions, ensuring that elements are only added or removed when valid. The `displayStack()` function effectively prints all elements from top to bottom, giving a clear view of the stack's current state. It could be improved by adding proper breaks in the switch statement and using local variables. At last, we managed to finish the displaytask function with the help of our instructor.

ILO B2 OUTPUT:

main.cpp | stacklist.h | Untitled-1

ILOB2 > main.cpp > main()

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include "stacklist.h"

int main(){

    std::cout<< "------ PUSHING 'c' 'p' 'e' ONTO THE STACK ------ \n";

    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top shpuld be 'e'
    displayAll<char>();
    std::cout << std::endl;

    std::cout<<"\n ------ Popping all the elements ------- \n";

    while(!isEmpty<char>()){
        std::cout<<" Popped:"<< pop<char>() <<std::endl;
    }

    std::cout<<"\n ------ Testing the Underflow ------ \n";
    pop<char>();

    return 0;
}
```

main.cpp | stacklist.h | Untitled-1

ILOB2 > stacklist.h > Top<T>()

```cpp
//Aguila, Daniel C. CPE21S1
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;     //VALUE THAT WE NEED STORE
    Node* next; //POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;
//operations for the stack


//push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    //newnode points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

```

main.cpp | stacklist.h | Untitled-1

ILOB2 > stacklist.h > Top<T>()

```
2   #ifndef STACK_LIST_H
21  void push(T newData){
28      head<T> = newNode;
29  }
30
31  //pop
32  template<typename T>
33  T pop(){
34      //check if the stack is empty;
35      if(head<T> == nullptr){
36          std::cout << "Stack Overflow. "<< std::endl;
37      }
38
39      //save the current top node
40      Node<T>* temp = head<T>;
41      //store its data
42      T temVal = temp->data;
43      //move the head to the next node
44      head<T> = head<T>->next;
45      //free the removed node
46      delete temp;
47      //return the removed value
48      return temVal;
49  }
50
51  //top
52  template<typename T>
53  void Top(){
54      //error catching
55      if(head<T> == nullptr){
56          std::cout<<"Stack undeflow "<<std::endl;
57          return;
58      }
59      //display the Top node
60      std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
61  }
62
63  //isEmpty
64  template<typename T>
65  bool isEmpty(){
66      return head <T> == nullptr;
67  }
```

main.cpp | stacklist.h | Untitled-1

ILOB2 > stacklist.h > Top<T>()

```
2   #ifndef STACK_LIST_H
53  void Top(){
59      //display the top node
60      std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
61  }
62
63  //isEmpty
64  template<typename T>
65  bool isEmpty(){
66      return head <T> == nullptr;
67  }
68
69  // displayAll
70  template <typename T>
71  void displayAll() {
72      if (head<T> == nullptr) {
73          std::cout << "Stack is empty." << std::endl;
74          return;
75      }
76      Node<T>* temp = head<T>;
77      while (temp != nullptr) {
78          std::cout << temp->data << "\n";
79          temp = temp->next;
80      }
81      std::cout << std::endl;
82  }
83
84  #endif
85
86
```

ILO B2 OUTPUT:

```
PS C:\Users\TIPQC\Documents\H0A4P1> cd "c:\Users\TIPQC\Documents\H0A4P1\ILOB2\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
------ PUSHING 'c' 'p' 'e' ONTO THE STACK ------
Top of Stack: e
e
p
c



 ------ Popping all the elements -------
 Popped:e
 Popped:p
 Popped:c

 ------ Testing the Underflow ------
Stack Overflow.
PS C:\Users\TIPQC\Documents\H0A4P1\ILOB2>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C CODE:

iloc.cpp

ILOC > iloc.cpp > areSymbolsBalanced(const std::string &, int)

```cpp
// Aguila, Daniel C. CPE21S1
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

bool isEmpty() { return top == -1; }
bool isFull(int maxSize) { return top == maxSize - 1; }

void push(char c, int maxSize) {
    if (isFull(maxSize)) {
        std::cout << "Stack overflow.\n";
        return;
    }
    stack[++top] = c;
}

char pop() {
    if (isEmpty()) {
        std::cout << "Stack underflow.\n";
        return '\0';
    }
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.